Southern New Hampshire University

**Runtime Analysis**

This section compares the runtime complexity of the three candidate data structures (vector, hash table, and binary search tree) for the primary operations required by the advising application. The analysis evaluates each structure based on loading course data, searching for a course, and printing the course list to determine the most appropriate overall implementation. Tables 1 through 3 summarize the worst-case runtime analysis of loading course data into each candidate data structure.

**Table 1. Vector Runtime Analysis**

| Code | Line Cost | # Times Executes | Total Cost |
|---|---|---|---|
| Create empty vector named courses | 1 | 1 | 1 |
| Open input file | 1 | 1 | 1 |
| Read one line | 1 | n | n |
| Split line at each comma | 1 | n | n |
| Create Course object | 1 | n | n |
| Add prerequisites | 1 | n | n |
| Add course to vector | 1 | n | n |
| **Total Cost** | | | **5n + 2** |
| **Runtime** | | | **O(n)** |

**Table 2. Hash Table Runtime Analysis**

| Code | Line Cost | # Times Executes | Total Cost |
|---|---|---|---|
| Create empty hash table named courses | 1 | 1 | 1 |
| Open input file | 1 | 1 | 1 |
| Read one line | 1 | n | n |
| Split line at each comma | 1 | n | n |
| Create Course object | 1 | n | n |
| Add prerequisites | 1 | n | n |
| Hash the course number | 1 | n | n |
| Insert course into hash table | n (worst case) | n | $n^2$ |
| **Total Cost** | | | **$n^2 + 5n + 2$** |
| **Runtime** | | | **$O(n^2)$** |

**Table 3. Binary Search Tree Runtime Analysis**

| Code | Line Cost | # Times Executes | Total Cost |
|---|---|---|---|
| Create empty binary search tree named courses | 1 | 1 | 1 |
| Open input file | 1 | 1 | 1 |
| Read one line | 1 | n | n |
| Split line at each comma | 1 | n | n |
| Create Course object | 1 | n | n |
| Add prerequisites | 1 | n | n |
| Insert course into binary search tree | n (worst case) | n | $n^2$ |
| **Total Cost** | | | **$n^2 + 4n + 2$** |
| **Runtime** | | | **$O(n^2)$** |

**Table 4. Big-O Runtime Comparison of Candidate Data Structures**

| Operation | Vector | Hash Table | Binary Search Tree |
|---|---|---|---|
| Load Data | O(n) | $O(n^2)$ | $O(n^2)$ |
| Search Course | O(n) | Average: O(1)<br>Worst: O(n) | Average: O(log n)<br>Worst: O(n) |
| Print Course List | O(n log n)<br>(Sort + Print) | O(n log n)<br>(Sort + Print) | O(n)<br>(In-Order Traversal) |

**Table 4** compares the runtime complexity of the three candidate data structures for the application's primary operations. Although the hash table provides the fastest average-case search performance, it does not maintain sorted order. The vector is simple to implement but requires linear searching and sorting before displaying courses. The binary search tree balances efficient searching with naturally ordered output through in-order traversal.

Southern New Hampshire University

**Table 5. Advantages and Disadvantages of the Data Structures**

| Data Structure | Advantages | Disadvantages |
|---|---|---|
| Vector | Simple implementation, low overhead | Linear searches; requires sorting before printing |
| Hash Table | Extremely fast average-case lookup | Does not maintain sorted order |
| Binary Search Tree | Maintains sorted order; efficient searching | More complex implementation |

**Table 5** summarizes the primary advantages and disadvantages of each candidate data structure. While the hash table provides the fastest average-case search performance, the binary search tree better satisfies the application's overall requirements because it maintains sorted order while still providing efficient search performance.

**Recommendation**

Although the hash table provides the fastest average-case lookup performance, it does not preserve sorted order, and therefore requires additional processing before displaying courses alphabetically. The vector implementation is straightforward but relies on linear searches and sorting before printing course information. The binary search tree provides efficient searching while naturally maintaining courses in alphanumeric order through in-order traversal. Because the advising application requires both efficient course lookups and alphabetically ordered output, the binary search tree is the most appropriate overall solution.